API REST con Node.js y MongoDB — Evaluación de MVP

1. Objetivo de la evaluación

Evaluar la capacidad del grupo para diseñar, implementar, documentar y sustentar un Producto Mínimo Viable (MVP) mediante una API REST desarrollada en Node.js, conectada a MongoDB. Cada funcionalidad debe estar asociada a una historia de usuario y demostrarse mediante Thunder Client o Postman

2. Entrega y publicación del proyecto

- El grupo debe publicar el proyecto en un repositorio público de GitHub.
- El grupo debe enviar el enlace del repositorio de GitHub al correo electrónico diego.prado.o@uniautonoma.edu.co
- La entrega debe realizarse antes de la fecha y hora límite establecida

3. Alcance del MVP

- El grupo debe entregar un MVP funcional que cubra las funcionalidades principales del proyecto planteado
- Cada funcionalidad debe estar asociada explícitamente a una historia de usuario
- El grupo debe sustentar 10 funcionalidades en máximo 5 minutos
- Se aceptan operaciones como consulta, registro, actualización, búsqueda, filtrado, validación de reglas de negocio o integración con otros recursos
- No se aceptan funcionalidades repetitivas o genéricas que no representen una necesidad real del proyecto

4. Historias de usuario

Cada funcionalidad debe estar relacionada con una historia de usuario con la siguiente estructura:

Como [rol o tipo de usuario], quiero [acción o necesidad], para [beneficio o valor obtenido]

- Las historias de usuario deben estar documentadas en el repositorio, por ejemplo en el README.md, en un archivo historias-usuarios.md o en la documentación del proyecto
- Durante la sustentación, el grupo debe mencionar la historia de usuario antes de demostrar cada funcionalidad
- La funcionalidad demostrada debe corresponder directamente con la historia de usuario enunciada

5. Requisitos de la API REST en Node.js

- El backend debe estar desarrollado en Node.js
- La API debe exponer endpoints REST correctamente estructurados
- Los endpoints deben utilizar métodos HTTP adecuados para cada operación
- La API debe responder en formato JSON
- La API debe utilizar códigos de estado HTTP coherentes con el resultado de cada operación
- El proyecto debe manejar errores básicos, evitando que una petición inválida detenga o rompa la aplicación

Ejemplo de estructura de endpoints

Operación	Método HTTP	Ejemplo de endpoint
Listar recursos	GET	/api/productos
Obtener un recurso	GET	/api/productos/:id
Crear recurso	POST	/api/productos
Actualizar recurso	PUT o PATCH	/api/productos/:id
Eliminar recurso	DELETE	/api/productos/:id

Códigos de estado HTTP esperados:

- 200 o 201: operación exitosa
- 400: datos inválidos o solicitud mal formada
- 404: recurso no encontrado
- 500: error interno del servidor

6. Conexión y persistencia en MongoDB

- La API debe conectarse a una base de datos MongoDB, local o en la nube
- Los datos creados, modificados o eliminados deben persistir en MongoDB
- No se aceptan datos almacenados únicamente en arreglos, variables en memoria o archivos locales
- Deben definirse colecciones y modelos o esquemas coherentes con el dominio del proyecto
- La cadena de conexión no debe exponer credenciales directamente en el código
- Se recomienda utilizar variables de entorno para almacenar la URI de conexión y credenciales

7. Demostración con Thunder Client o Postman

- El grupo debe demostrar cada funcionalidad utilizando Thunder Client o Postman
- En la demostración se debe evidenciar el método HTTP utilizado
- Se debe mostrar la URL o endpoint invocado

- Cuando aplique, se debe mostrar el cuerpo de la petición en formato JSON
- Se debe evidenciar la respuesta obtenida y el código de estado HTTP retornado
- Para funcionalidades de creación, actualización o eliminación, se debe verificar la persistencia del dato en MongoDB
- No se acepta únicamente mostrar el código fuente o explicar la funcionalidad sin ejecutarla

8. Sustentación oral

- La sustentación tiene una duración máxima de 5 minutos por grupo
- El grupo debe **presentar 10 funcionalidades**
- Cada funcionalidad debe estar asociada a una historia de usuario
- Por cada funcionalidad, el grupo debe

    1. Mencionar la historia de usuario
    2. Indicar el endpoint y el método HTTP utilizado
    3. Ejecutar la petición en Thunder Client o Postman
    4. Mostrar la respuesta obtenida
    5. Verificar la persistencia en MongoDB, cuando aplique

- Todos los integrantes del grupo deben participar en la sustentación
- Si el grupo no alcanza a demostrar las 10 funcionalidades en el tiempo establecido, solo se evaluarán las funcionalidades efectivamente demostradas

9. Criterios de evaluación

Criterio	Descripción	Puntaje
Publicación en GitHub	Repositorio público, completo, documentado y enlace enviado correctamente.	10%
MVP del proyecto	El conjunto de funcionalidades representa un producto mínimo viable coherente con el proyecto planteado.	15%
API REST en Node.js	Endpoints correctamente estructurados, métodos HTTP adecuados, respuestas JSON y manejo básico de errores.	20%
MongoDB	Conexión correcta, modelos o esquemas definidos y persistencia real de los datos.	15%
Historias de usuario	Cada funcionalidad está asociada a una historia de	10%

	usuario clara y pertinente.	
Demostración funcional	Uso correcto de Thunder Client o Postman y evidencia de funcionamiento de cada endpoint.	20%
Sustentación	Claridad, manejo del tiempo, participación del grupo y coherencia entre historia de usuario y funcionalidad.	10%
Total		100%

10. Condiciones de no aceptación

- El proyecto no está publicado en GitHub o el enlace enviado no es accesible
- La API no está desarrollada en Node.js
- La aplicación no se conecta a MongoDB
- Las funcionalidades no persisten los datos en la base de datos
- Las demostraciones no se realizan mediante Thunder Client o Postman
- Las funcionalidades se presentan sin mencionar la historia de usuario asociada
- El grupo no cumple con el límite de 5 minutos de sustentación
- El grupo no demuestra las 10 funcionalidades requeridas

11. Formato de registro de funcionalidades

El grupo puede utilizar la siguiente tabla para documentar las 10 funcionalidades y sus historias de usuario:

N.º	Historia de usuario	Funcionalidad	Endpoint y método HTTP	Resultado esperado
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				